

COMP 532

Machine Learning and Bioinspired Optimisation

Lecture 7 Reinforcement Learning



Meng Fang
University of Liverpool

Recap

- Reinforcement Learning
 - Solve the problem in Markov Decision Processes (MDPs)
 - Returns
 - Markov Property

Overview

- Problem Solving from Nature
- Reinforcement Learning (RL)
 - Roots of RL
 - Key features of RL
 - Elements of RL
 - Multi-armed bandits
 - Solve the problem in Markov Decision Processes (MDPs)
 - **Value function**
- Deep Learning
- Multi-Agent Learning
- Swarm Intelligence

Value Function

- The value of a state is the expected return starting from that state; depends on the agent's policy:

State-value function for policy π

$$V^\pi(s) = E_\pi\{G_t | s_t = s\} = E_\pi\left\{\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | s_t = s\right\}$$

- The value of taking an action in a state under policy π is the expected return starting from that state, taking that action, and thereafter following π :

Action-value function for policy π

$$Q^\pi(s, a) = E_\pi\{G_t | s_t = s, a_t = a\} = E_\pi\left\{\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | s_t = s, a_t = a\right\}$$

Value Function

- Return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k}$

- Value functions $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$
 $Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$

- Interpretation:
- V^{π} :how good is a state
- Q^{π} :how good is taking action a in s

Bellman Equation (Policy Version)

From:

- $G_t = R_{t+1} + \gamma G_{t+1}$

Take expectation:

- $V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$

Expanded form:

- $V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$

Bellman Equation (Policy Version)

Step 1 — Use linearity of conditional expectation

- $\mathbb{E}[X + Y \mid Z] = \mathbb{E}[X \mid Z] + \mathbb{E}[Y \mid Z]$
- So:
- $V^\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s]$
- Now we expand each term.

We have $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and $G_t = R_{t+1} + \gamma G_{t+1}$

Then $V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_t = s]$

For $\mathbb{E}_\pi[G_{t+1} \mid S_t = s]$

- According MDP, $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s'] = V^\pi(s')$
- $S_t = s$ is not S_{t+1}
- So we use tower property : $\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \mathbb{E}_\pi[\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] \mid S_t = s]$
- Then we get $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] = V^\pi(S_{t+1})$
- Therefore :
- $= \mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s]$

Bellman Equation (Policy Version)

Step 2 — Expand the first term (Reward)

Under policy π , the randomness comes from:

- The action $A_t \sim \pi(\cdot | s)$
- The transition $S_{t+1} \sim P(\cdot | sa)$

Therefore:

- $$\mathbb{E}_{\pi}[R_{t+1} | S_t = s] = \sum_a \pi(a | s) \sum_{s'} P_{ss'}^a R_{ss'}^a$$

Explanation:

- First average over possible actions
- Then average over possible next states
- Multiply by reward for that transition
- So the conditioning on $S_t = s$ determines:
- The action distribution
- The transition distribution

Bellman Equation (Policy Version)

- **Step 3 — Expand the second term (Next-state value)**

Now consider:

- $\mathbb{E}_{\pi}[V^{\pi}(S_{t+1}) \mid S_t = s]$

Again, under policy π :

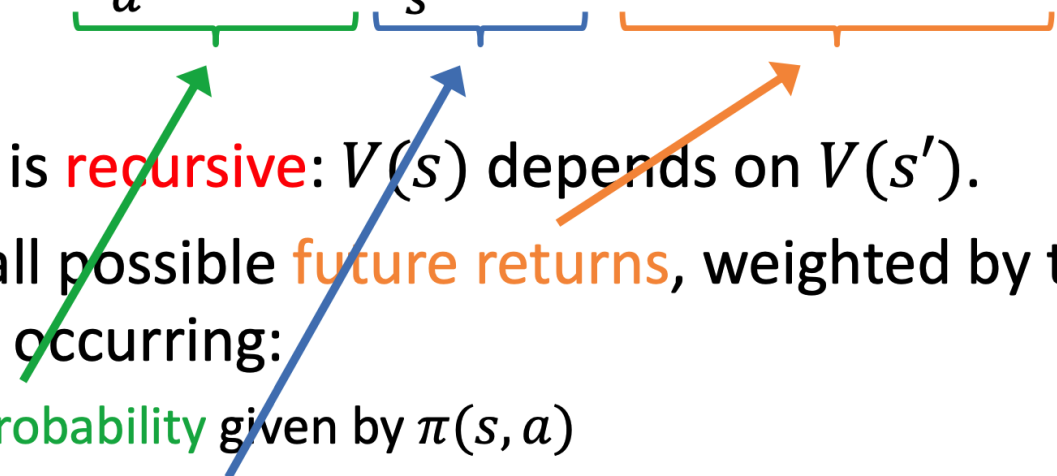
- $= \sum_a \pi(a \mid s) \sum_{s'} P_{ss'}^a V^{\pi}(s')$

Why?

- Because:
- If the next state turns out to be s' ,
its value is $V^{\pi}(s')$
- We average over all possible next states

Bellman Equation

- State value function:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$$
$$= \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$


- The equation is **recursive**: $V(s)$ depends on $V(s')$.
- It sums over all possible **future returns**, weighted by their probability of occurring:
 - the **action probability** given by $\pi(s, a)$
 - the **state transition probability** given by $P_{ss'}^a$

Bellman Equation: Intuition

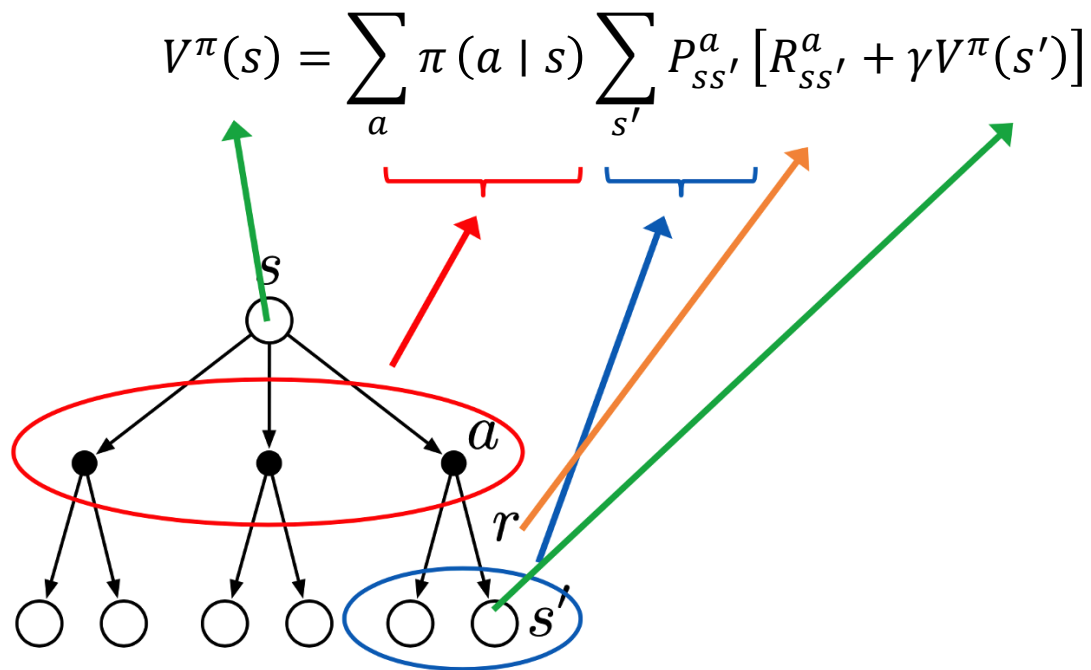
$$V^{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^{\pi}(s')]$$

The value of a state equals its immediate reward plus the discounted value of its successor states, averaged according to the policy and transition dynamics.

The Bellman equation characterises value as a **recursive** expectation: the value of a state equals the expected immediate reward plus the discounted value of successor states, averaged according to the policy and transition dynamics.

More on the Bellman Equation

- State value function:



Policy Evaluation (Algorithm)

Iterative update:

- $$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s'} P_{ss'}^a [R + \gamma V_k(s')]$$

Converges to V^π .

Optimal value function

Definition:

- $V^*(s) = \max_{\pi} V^{\pi}(s)$

Optimal policy may not be unique.

Optimal value function is unique.

Value Iteration

- $V_{k+1}(s) \leftarrow \max_a \sum_{s'} P_{ss'}^a [R + \gamma V_k(s')]$

As $k \rightarrow \infty$:

- $V_k \rightarrow V^*$

Bellman Optimality Equation

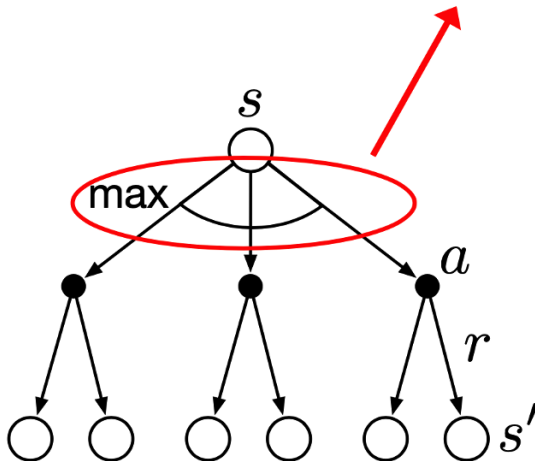
- Optimal value function

$$V^*(s) = \max_{\pi} V^{\pi}(s), \forall s \in S$$

- Optimal action-value function

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a), \forall s \in S, a \in A$$

$$V^*(s) = \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^*(s')]$$



How to use policy

Once V^* is known:

- $\pi^*(s) = \arg \max_a \sum_{s'} P_{ss'}^a [R + \gamma V^*(s')]$

Alternatively:

- $\pi^*(s) = \arg \max_a Q^*(s, a)$

Bellman Optimality for Q

- $Q^*(s, a) = \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma \max_{a'} Q^*(s', a')]$

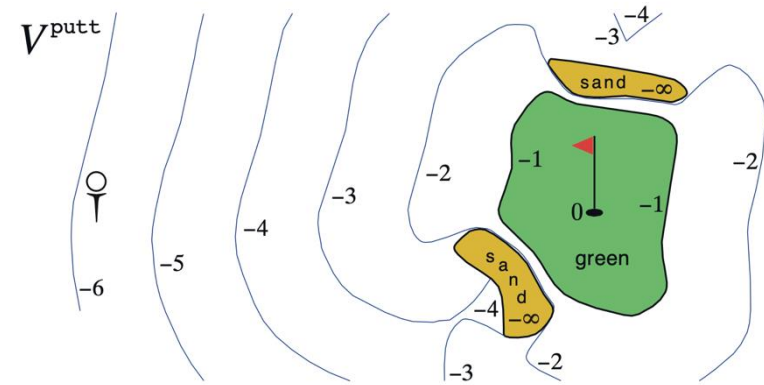
Policy Iteration

Algorithm:

- Policy Evaluation
 - Policy Improvement
-
- Policy improvement theorem guarantees monotonic improvement.

Example: Golf

- State is ball location
- Reward of -1 for each stroke until the ball is in the hole
- Value of a state?
- Actions:
 - putt (use putter)
 - driver (use driver)
- putt succeeds anywhere on the green



Compute:

For action a_1 :

- $$Q(s, a_1) = \sum_{s'} P_{ss'}^{a_1} [R + \gamma V(s')]$$

For action a_2 :

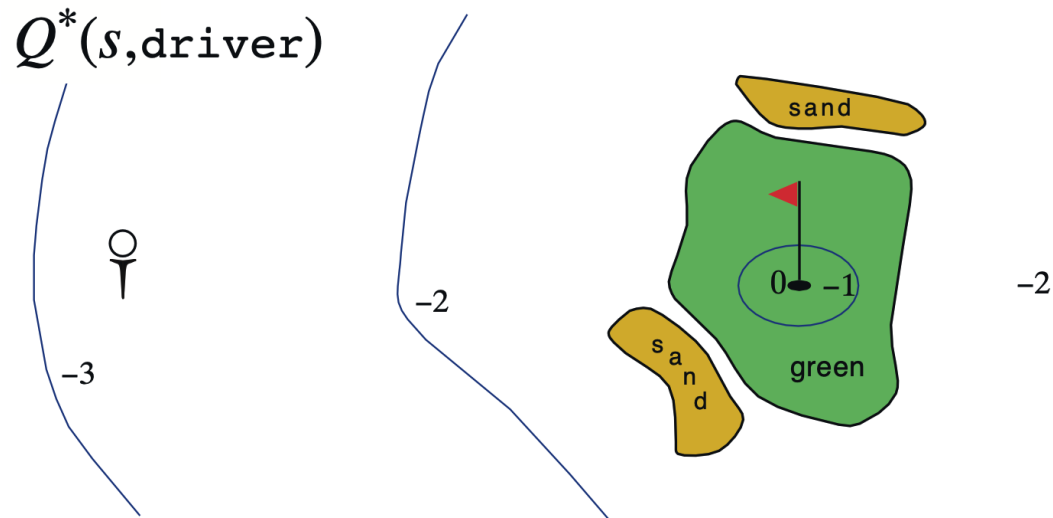
- Compute same.

Then:

- $$V^*(s) = \max\{Q(s, a_1), Q(s, a_2)\}$$

Example: Optimal Value Function for Golf

- We can hit the ball farther with driver than with putter, but with less accuracy
- $Q^*(s, \text{driver})$ gives the value for using driver first, then using whichever actions are best



Why this is “Dynamic Programming” (DP)

Recursive decomposition (Bellman backup)

- $V(s) = \mathbb{E}[R + \gamma V(S') \mid S = s]$
- Value of a state reduces to values of successor states.

Greedy improvement $\Rightarrow$ Policy Iteration

- $\pi'(s) = \arg \max_a \mathbb{E}[R' + \gamma V^\pi(S') \mid s, a] \Rightarrow V^{\pi'} \geq V^\pi$

Therefore: DP = repeated Bellman backups (planning with known P, R).

Summary

Bellman Equation

DP requires:

- Full model P, R
- It computes exact expectations over next-state distributions rather than estimating them from samples.

Next lecture:

- Monte Carlo
- TD Learning
- Q-learning